SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO5: *Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas – DataFrame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)*

Assessment Tool Weightage: 40%

QUESTIONS: 03 **Total Marks:25**

Annexure A

Constraint Based Problem Solving

Code of Conduct:

I Jeevan Raaja S (192421005) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Total*
Weightage	*30%*	*25%*	*15%*	*15%*	*15%*	
Q1						
Q2						

Signature of the student: Jeevan Raaja S ***Total Marks :***

Annexure A

1. You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

2. A system with limited memory must process a massive numerical dataset using NumPy arrays. You are required to perform mathematical operations, random sampling, and statistical analysis without exceeding memory limits. Propose a solution that uses efficient array creation, slicing, and in-place operations. Discuss how indexing and aggregation can be optimized to minimize memory usage, and explain debugging strategies if memory overflow or performance bottlenecks occur.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**15**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

1.A **constraint-based data cleaning pipeline** applies predefined rules to the patient dataset so that the final data is complete, valid, consistent, and suitable for analysis. **Pandas** can be used for data loading, indexing, filtering, and categorical processing, while **NumPy** can be used for numerical operations and validation.

1. Load the dataset

First, read the CSV file using Pandas.

```
import pandas as pd
import numpy as np

df = pd.read_csv("patient_records.csv")

print(df.head())
print(df.info())
```

This helps identify the columns, data types, and missing values.

2. Handle missing values

Missing values can be detected using `isnull()`.

```
print(df.isnull().sum())
```

Numerical missing values can be replaced with the **median**, because the median is less affected by outliers.

```
num_cols = df.select_dtypes(include=np.number).columns

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())
```

For categorical columns, the missing value can be replaced by the **mode**.

```
cat_cols = df.select_dtypes(include="object").columns

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Finally, verify that no missing values remain:

```
print(df.isnull().sum().sum())
```

The result should be:

0

3. Enforce medically valid numerical ranges

Constraints are applied to ensure that numerical values are within reasonable medical limits.

For example:

```
df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]

df = df[(df["HeartRate"] >= 30) & (df["HeartRate"] <= 220)]

df = df[(df["Temperature"] >= 35) & (df["Temperature"] <= 42)]
```

The **filtering operation** removes records that violate the specified constraints.

For a large dataset, these constraints should be defined according to the medical requirements of the particular dataset rather than arbitrarily.

4. Detect and handle outliers

Outliers can be identified using NumPy and statistical measures such as the **IQR (Interquartile Range)**.

```
Q1 = df["HeartRate"].quantile(0.25)
Q3 = df["HeartRate"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df = df[(df["HeartRate"] >= lower) &
        (df["HeartRate"] <= upper)]
```

This prevents extreme values from negatively affecting statistical analysis.

5. Standardize categorical fields

Categorical values may have inconsistent formats such as:

Male
male
M
MALE

They can be standardized using string operations and mapping.

```
df["Gender"] = df["Gender"].str.strip().str.lower()
```

```
df["Gender"] = df["Gender"].replace({
    "m": "male",
    "f": "female"
})
```

Now the categorical values follow a consistent format.

6. Use indexing

Indexing allows specific rows and columns to be accessed efficiently.

```
print(df.loc[0:5, ["Age", "Gender", "HeartRate"]])
```

Boolean indexing can also identify records satisfying a constraint:

```
healthy_age = df.loc[df["Age"].between(18, 65)]
```

Thus, indexing helps inspect and validate specific portions of the dataset without modifying unrelated records.

7. Use filtering

Filtering ensures that only records satisfying the constraints are retained.

```
valid_records = df[
    (df["Age"].between(0, 120)) &
    (df["HeartRate"].between(30, 220)) &
    (df["Temperature"].between(35, 42))
]
```

This directly applies the constraints to the dataset.

8. Use aggregation functions

Aggregation functions help check the quality of the cleaned data.

```
print(df[["Age", "HeartRate", "Temperature"]].agg(
    ["min", "max", "mean", "median"]
))
```

Functions such as `min()`, `max()`, `mean()`, and `median()` can verify whether numerical values remain within expected ranges.

Grouping can also be used to analyze patient records by category:

```
print(df.groupby("Gender")["HeartRate"].mean())
```

9. Final validation

After cleaning, the constraints should be checked again.

```
print("Missing values:", df.isnull().sum().sum())
```

```
print("Minimum Age:", df["Age"].min())
print("Maximum Age:", df["Age"].max())
```

```
print(df["Gender"].unique())
```

The final dataset should satisfy:

Constraint	Method Used
No missing values	`fillna()`, `isnull()`
Valid numerical values	Boolean filtering
Outlier control	IQR
Standard categorical values	`str.strip()`, `str.lower()`, `replace()`
Data inspection	Indexing with `loc`
Constraint enforcement	Boolean filtering
Data validation	Aggregation functions

Conclusion

The proposed pipeline combines **Pandas and NumPy** to clean patient health records systematically. Missing values are handled using statistical replacement, numerical constraints are enforced through filtering, outliers are detected using IQR, and categorical fields are standardized. **Indexing** provides controlled access to data, while **filtering** enforces constraints and **aggregation** verifies the quality of the cleaned dataset. This approach preserves maximum data integrity while producing a consistent and analysis-ready dataset.

2.When processing a massive numerical dataset with limited memory, the main objective is to **reduce unnecessary memory allocation and avoid creating duplicate arrays**. NumPy provides efficient techniques such as appropriate data types, slicing, in-place operations, indexing, and aggregation.

1. Efficient Array Creation

The data type should be selected according to the required precision. Using `float32` instead of `float64` can significantly reduce memory usage.

```
import numpy as np

data = np.random.rand(10000000).astype(np.float32)

print(data.nbytes / (1024 ** 2), "MB")
```

Using an appropriate data type prevents unnecessary memory consumption.

For integer data, smaller integer types can also be used when the range permits:

data = np.arange(1000000, dtype=np.int32)

2. Use Slicing Instead of Copying

NumPy slicing generally creates a **view** of the original array instead of a separate copy.

subset = data[100000:200000]

This is memory efficient because `subset` refers to the existing data.

Avoid unnecessary copying:

subset = data[100000:200000].copy()

A copy requires additional memory, so it should only be used when modification of the original array must be avoided.

3. Perform Mathematical Operations In-Place

In-place operations modify the existing array instead of creating a new array.

data *= 2
data += 10
data /= 5

Instead of:

data = data * 2
data = data + 10

The in-place approach reduces temporary arrays and therefore reduces memory usage.

For more control, NumPy's `out` parameter can also be used:

np.multiply(data, 2, out=data)

4. Efficient Random Sampling

Instead of creating a large additional array, generate only the required sample.

```
rng = np.random.default_rng(42)

sample_indices = rng.choice(
    len(data),
    size=10000,
    replace=False
)

sample = data[sample_indices]
```

This allows statistical analysis to be performed on a representative sample when processing the complete dataset is unnecessary.

5. Efficient Indexing

Boolean indexing is useful for selecting required values:

```
positive = data[data > 0.5]
```

However, boolean indexing creates a new array. Therefore, for very large datasets, it should be used carefully.

When possible, use slices:

```
subset = data[100000:200000]
```

Slicing is preferable when a continuous range of elements is required because it normally provides a view without allocating another full array.

6. Optimize Aggregation

Statistical operations can often be performed directly on the original array without storing intermediate results.

```
mean_value = np.mean(data)
```

```
minimum = np.min(data)
maximum = np.max(data)
standard_deviation = np.std(data)
```

Multiple statistics can be calculated as required:

```
print("Mean:", np.mean(data))
print("Minimum:", np.min(data))
print("Maximum:", np.max(data))
```

This avoids creating unnecessary intermediate arrays.

For grouped or chunk-based processing, the dataset can be processed in smaller portions.

```
chunk_size = 1000000


total = 0
count = 0


for start in range(0, len(data), chunk_size):
    chunk = data[start:start + chunk_size]
    total += np.sum(chunk)
    count += len(chunk)


mean_value = total / count
print("Mean:", mean_value)
```

This approach is useful when the complete dataset is too large to process simultaneously.

7. Memory-Efficient Processing Strategy

The complete workflow can be represented as:

Large Dataset → Efficient Data Type → Slicing/Chunks → In-Place Operations → Sampling → Aggregation → Final Statistics

The important techniques are:

Technique	Purpose

`dtype=np.float32`	Reduce memory per element
Slicing	Access portions without unnecessary copies
In-place operations	Avoid temporary arrays
Random sampling	Process only required observations
Direct aggregation	Avoid intermediate arrays
Chunk processing	Handle datasets larger than available memory

8. Debugging Memory Overflow

If the program produces a memory error, first check the size of the array.

print(data.nbytes / (1024 ** 2), "MB")

Check the data type:

print(data.dtype)

The shape can also be inspected:

print(data.shape)

If the array is unnecessarily large, use a smaller data type:

data = data.astype(np.float32)

Also identify operations that create copies or temporary arrays.

For example, instead of creating several intermediate arrays:

result = (data * 2) + 10

use in-place operations where appropriate:

data *= 2
data += 10

9. Debugging Performance Bottlenecks

Performance can be checked by measuring execution time.

```
import time

start = time.time()

mean_value = np.mean(data)

end = time.time()

print("Execution time:", end - start)
```

If processing is slow:

- Prefer NumPy vectorized operations over Python loops.
- Use slicing and chunk processing.
- Avoid unnecessary `.copy()`.
- Use appropriate data types.
- Avoid repeatedly allocating large arrays.
- Perform aggregation directly whenever possible.

Conclusion

For a massive numerical dataset on a memory-limited system, NumPy should be used with **memory-efficient data types, slicing, in-place operations, efficient random sampling, optimized indexing, and direct aggregation**. If the dataset is too large, **chunk-based processing** allows it to be analyzed without loading or creating unnecessary copies of the complete data. Memory usage can be debugged using `nbytes`, `dtype`, and `shape`, while performance can be improved through vectorization and avoiding unnecessary array creation. This approach satisfies the memory and performance constraints while maintaining efficient numerical processing.